

Deployment Strategies and Model Serialization for ML Models

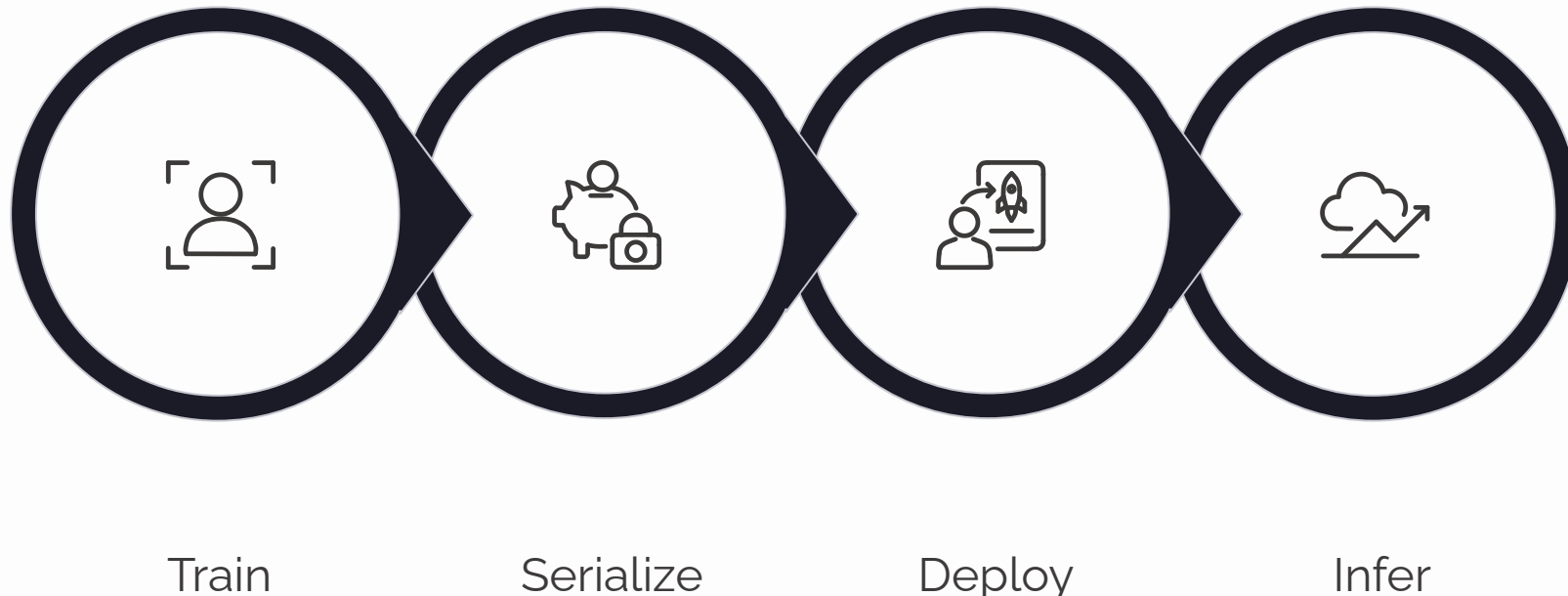
Once a machine learning model is trained and evaluated, the next question is: **How do we make it available for real users or applications?** This is called **Model Deployment** – and a critical part of it is **Model Serialization**, saving the trained model so it can be loaded and used later without retraining.

Dr. Anilkumar Vishwanath Brahmane
Computer Engineering Department
Sanjivani College of Engineering



The Complete ML Lifecycle

Deployment is the final bridge between a trained model and real-world use. The full lifecycle flows from problem definition through training, serialization, and into production.



The key insight: **Training** → **Serialization** → **Deployment** is the production pipeline every ML engineer must master.

Model Serialization & Deserialization

Serialization converts a trained model in memory into a file saved on disk or cloud. **Deserialization** loads that file back into memory for predictions — no retraining needed.

What Gets Stored?

- Model architecture
- Learned weights & biases
- Model configuration
- Optimizer state
- Preprocessing metadata

Framework Formats

- **Scikit-learn:** joblib, pickle (.pkl)
- **TensorFlow/Keras:** .keras, SavedModel
- **PyTorch:** state_dict, checkpoint (.pth)
- **Cross-framework:** ONNX (.onnx)
- **Mobile/Edge:** TensorFlow Lite (.tflite)

📄 **Best Practice:** Always save preprocessing (e.g., StandardScaler) together with the model using Scikit-learn **Pipeline** — so preprocessing and model travel together and predictions remain reliable.

Deployment Strategies

After serialization, we decide **where and how** the model runs. Each strategy suits different use cases and infrastructure needs.

1

Local

Model runs on the same machine. Simple, no internet needed. Ideal for prototypes and college projects.

2

Web / REST API

Model served via Flask or FastAPI. Multiple clients send HTTP requests and receive JSON predictions.

3

Cloud

Deployed on AWS, Azure, or Google Cloud. Scalable to millions of users on demand.

4

Edge

Model runs on smartphones, cameras, or IoT devices — close to the data source, with low latency.

5

Batch

Processes large record sets together (e.g., nightly churn prediction for 100,000 customers).

6

Real-Time

Instant predictions per request (e.g., credit card fraud detection in milliseconds).

Production Challenges & Model Drift

A model working in a notebook is **not automatically production-ready**. Real deployment demands attention to several critical concerns.

Versioning

Track which model version (v1, v2, v3) is currently deployed.

Dependencies

Match Python, Scikit-learn, TensorFlow, PyTorch, NumPy versions in production.

Input Validation

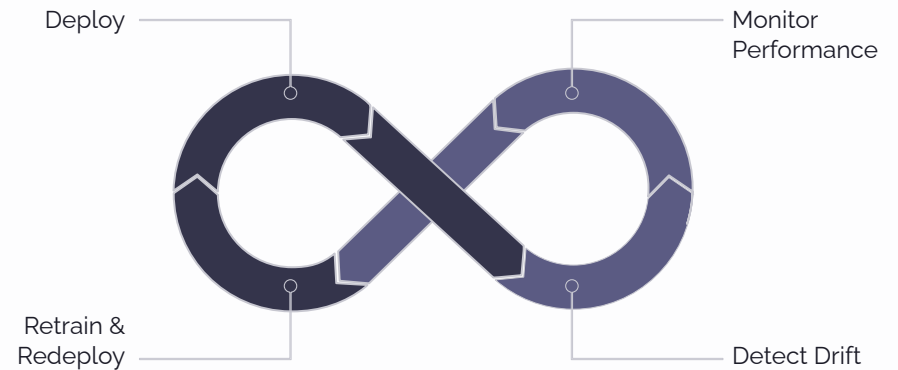
Verify incoming data is valid before passing to the model.

Scalability

Can the system handle 10, 10,000, or 1 million users?

Model Drift

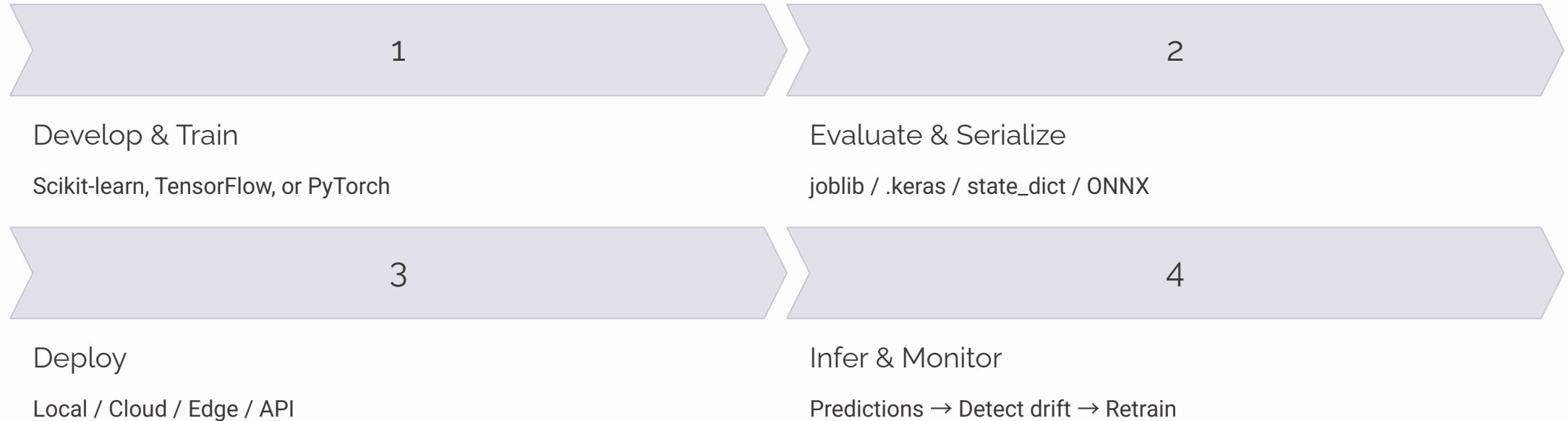
A model trained on 2024 data may become less accurate as real-world behavior changes. The solution is a continuous cycle:



Security: Never load pickle/joblib files from untrusted sources — malicious serialized objects can execute arbitrary code during loading.

The Complete Production Cycle

The full ML production pipeline – from development to ongoing monitoring – can be summarized in one flow every student should internalize:



One-line summary: Model development creates and trains the ML model; serialization saves it; deployment makes that saved model available to applications, users, cloud systems, or edge devices for real-world predictions.